Actual HW3

Sam Olson

Q1

Using the mixture model (1) and (2) fit the data in Data Set 1 and Data Set 2 separately using maximum likelihood. Produce approximate 95% confidence intervals for the modes of the distributions – these are the value of the parameter ξ in (2). Conduct any additional analysis needed to construct a likelihood ratio test of a reduced model in which the parameters ξ and θ are identical for both data sets against a full model in which those parameters differ between the two heights. Graph the extreme value density or densities for whichever model (full or reduced) you select.

Data model:

$$f(y \mid \mu_i) = \frac{1}{(2\pi)^{1/2}} \exp\left[-\frac{1}{2}(y-\mu_i)^2\right]. \tag{1}$$

Mixing distribution:

$$g(\mu \mid \xi, \theta) = \frac{1}{\theta} \exp\left(-\frac{\mu-\xi}{\theta}\right) \exp\left[-\exp\left(-\frac{\mu-\xi}{\theta}\right)\right], \qquad \theta > 0. \tag{2}$$

A:

LIKELIHOOD DIRECT

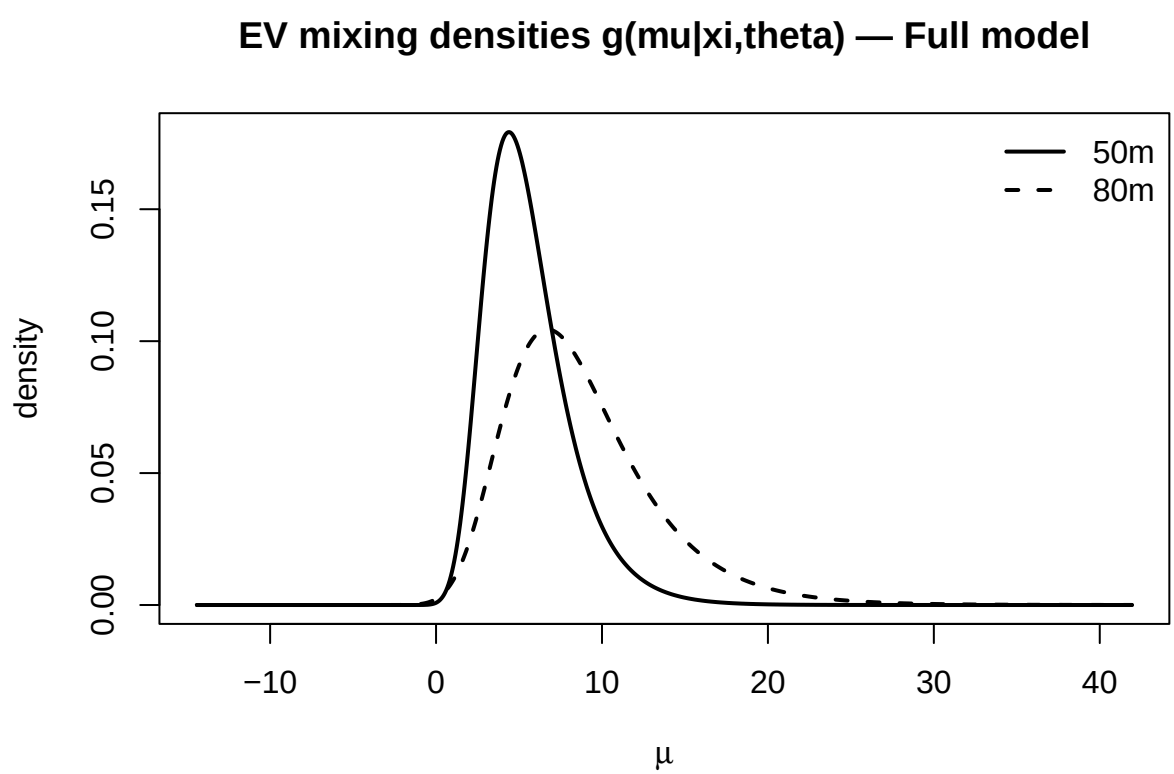
```
##
## --- MLE (50m) ---

## [1] 4.392454

## Wald CI xi (50m):

## [1] 3.718789 5.066119

##
## --- MLE (80m) ---

## [1] 6.716659

## Wald CI xi (80m):

## [1] 5.645922 7.787395
```

PROFILE

```
##
## Profile-LR CI xi (50m):

## [1] 3.700317 5.074705

##
## Profile-LR CI xi (80m):

## [1] 5.632896 7.822622
```

LRT

```
##
## --- LRT (Full vs Reduced) ---

##           LR           df      p_value
## 1.857906e+01 2.000000e+00 9.238627e-05
```



Q2

In (2), $E(\mu) \approx \xi + 0.577\theta$, where 0.577 is Euler's constant (don't confuse with Euler's number). Using this expression, produce maximum likelihood estimate or estimates and 95% confidence interval or confidence intervals for the expected true maximum wind speed from your final model. Note that this will also be the expected observed maximum wind speed.

A:

```
##
## --- Q2: E(mu) ---

## $eta_hat
## [1] 5.57714
##
## $se
## [1] 0.3956778
##
## $ci95
## [1] 4.801626 6.352655

## $eta_hat
## [1] 8.74983
##
## $se
## [1] 0.6509698
##
## $ci95
## [1]  7.473953 10.025708
```

Q3

Find a maximum likelihood estimate or estimates (depending on your final model) of the median daily maximum wind speed or speeds. Compute the associated 95% approximate confidence interval or intervals.

A:

```
##
## --- Q3: Median(Y) ---

## $median_hat
## [1] 5.207891
##
## $se
## [1] 0.3674746
##
## $ci95
## [1] 4.487654 5.928128
##
## $grad
## [1] 1.0000000 0.7091163

## $median_hat
## [1] 8.048726
##
## $se
## [1] 0.6009588
##
## $ci95
## [1] 6.870869 9.226584
##
## $grad
## [1] 1.000000 1.254696
```

Q4

Find a maximum likelihood estimate or estimates (depending on your final model) for the probability that true daily maximum wind speed is between 4 and 17 m/s. Compute 95% approximate confidence intervals for the estimate or estimates.

A

```
##
## --- Q4: P(4 <= mu <= 17) ---
```

```
## $p_hat
## [1] 0.6998708
##
## $se
## [1] 0.05975642
##
## $ci95
## [1] 0.5827504 0.8169912
##
## $grad
## [1]  0.17473516 -0.08215758
```

```
## $p_hat
## [1] 0.8324267
##
## $se
## [1] 0.0456608
##
## $ci95
## [1] 0.7429332 0.9219203
##
## $grad
## [1]  0.05611005 -0.34113449
```

Q5

What would you conclude about the question of whether there is or is not a benefit to placing turbine hubs at either 50 or 80 m in height?

A:

In Q1, the likelihood ratio test provided evidence that the wind speed distributions at 50m and 80m differ, leading us to select the full model with separate parameter estimates for each height. The estimated parameters imply systematically different wind speed behavior across heights.

In particular, the estimated probability that true daily maximum wind speed lies in the operationally relevant range of 4–17 m/s is larger at 80m than at 50m. The corresponding confidence intervals indicate that this difference is not attributable to sampling variability alone. Additionally, the estimated mean and median daily maximum wind speeds are higher at 80m.

Taken together, these results suggest that, under the fitted model and the conditions represented in the observed data, placing turbine hubs at 80m is associated with more favorable wind speed characteristics for turbine operation. This conclusion is conditional on the validity of the model assumptions and on future wind conditions being similar to those reflected in the data used for this analysis.

Q6

As an exercise in learning about statistical procedures, your friend would like to see a Monte Carlo approximation to the expected true wind speed for the two heights separately. Produce such approximations along with 95% Monte Carlo intervals. Your friend understands intuitively that these approximations and the maximum likelihood estimates of Question 2 should have the same or nearly the same values, but marvels at the fact that the intervals associated with these approximations are so much more narrow than those computed using the asymptotic theory of likelihood estimation in Question 2. Explain why this is the case and what it means.

A:

Monte Carlo

```
##
## --- Q6: Monte Carlo E(mu) ---

## [1] 5.582784

## [1] 5.546062 5.619506

## [1] 8.7058

## [1] 8.643696 8.767904
```

Comparison - Concise

The Monte Carlo (MC) intervals are much narrower because they measure only simulation (numerical) error conditional on the estimated parameters $(\hat{\xi}, \hat{\theta})$, whereas the asymptotic confidence intervals in Question 2 account for statistical uncertainty in the joint estimation of (ξ, θ) from finite data.

Comparison - More Details

For each height, we simulated

$$\mu^{(1)}, \ldots, \mu^{(M)} \sim \mathrm{EV}(\hat{\xi}, \hat{\theta})$$

and estimated the expected true wind speed by

$$\widehat{E}_{MC}(\mu) = \bar{\mu}_M = \frac{1}{M} \sum_{m=1}^{M} \mu^{(m)}.$$

By the Central Limit Theorem for simulation,

$$\bar{\mu}_M \approx \mathcal{N}\left(E(\mu), \frac{\mathrm{Var}(\mu)}{M}\right),$$

so an approximate 95% Monte Carlo interval is

$$\bar{\mu}_M \pm 1.96 \frac{s_M}{\sqrt{M}}.$$

The crucial point is that this interval treats $(\hat{\xi}, \hat{\theta})$ as fixed and quantifies only Monte Carlo approximation error, which decreases at rate $1/\sqrt{M}$ and can be made arbitrarily small by increasing M.

In contrast, the confidence interval in Question 2 is based on asymptotic likelihood theory for

$$E(\mu) = \xi + \gamma\theta, \qquad \gamma \approx 0.577,$$

and reflects uncertainty in estimating (ξ, θ) from $n = 50$ observations. That parameter uncertainty propagates into uncertainty about $E(\mu)$.

Thus, the Monte Carlo intervals are narrow because they measure numerical precision, not inferential uncertainty. The wider intervals from Question 2 are the appropriate measures of statistical uncertainty about the expected wind speed. To obtain a simulation-based interval comparable to Question 2, one would need to incorporate parameter uncertainty (e.g., via a parametric bootstrap), rather than conditioning on fixed MLEs.

Appendix

```r
# ============================================================
# APPENDIX: All-in-one code chunk for PS3 wind mixture model
# Normal-EV (right-skew Gumbel) mixture; MLE via newtraph
# + Wald and profile-LR CIs for xi
# + LRT (full vs reduced) across heights
# + Q2-Q4 delta-method CIs
# + Q6 Monte Carlo approximation + MC intervals
# ============================================================

library(tidyverse)
library(numDeriv)

# ---- Source helper routines (provide these files or edit paths) ----
source("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/newtraph.txt")
source("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/bisect.txt")

# ---- Load data ----
m50 <- read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat1.txt",
                  header = TRUE)
m80 <- read.table("C:/Users/samue/OneDrive/Desktop/Iowa_State_PS/STAT_6010/PS3/winddat2.txt",
                  header = TRUE)
y50 <- as.numeric(m50$y)
y80 <- as.numeric(m80$y)

# ============================================================
# Core model components
# ============================================================

# Stable marginal density m(y | xi, theta) using z = (mu - xi)/theta
m_y_stable <- function(y, xi, theta, zL = -100, zU = 100) {
  integrand <- function(z) {
    # log integrand = log N(y | xi + theta z, 1) + log{exp(-z - exp(-z))}
    log_int <- dnorm(y, mean = xi + theta * z, sd = 1, log = TRUE) - z - exp(-z)
    exp(log_int)
  }

  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value
  pmax(val, .Machine$double.xmin)
}

log_m_vec_stable <- function(y, xi, theta, zL = -100, zU = 100) {
  vapply(y, function(yy) log(m_y_stable(yy, xi, theta, zL, zU)), numeric(1))
}

# log-likelihood for one dataset; par = (xi, logtheta)
logLik_one <- function(par, y, zL = -100, zU = 100) {
  xi <- par[1]
  theta <- exp(par[2])
  sum(log_m_vec_stable(y, xi, theta, zL = zL, zU = zU))
}
```

```r
# starting values (moment matching)
ev_start <- function(y) {
  gamma <- 0.5772156649015329
  v <- var(y)
  theta0 <- sqrt(pmax((v - 1) * 6 / pi^2, 1e-6))  # Var(Y)=pi^2/6*theta^2 + 1
  xi0 <- mean(y) - gamma * theta0                 # E(Y)=xi + gamma*theta
  c(xi0, log(theta0))
}

# newtraph glue: ders(par, dat) -> list(ll, grad, Hessian)
make_ders_one <- function(zL = -100, zU = 100) {
  function(par, dat) {
    f <- function(pp) logLik_one(pp, y = dat, zL = zL, zU = zU)
    ll <- f(par)
    g  <- numDeriv::grad(f, par)
    H  <- numDeriv::hessian(f, par)
    list(ll, g, H)
  }
}

# fit one dataset using newtraph
fit_one_newtraph <- function(y, zL = -100, zU = 100) {
  start <- ev_start(y)
  ders  <- make_ders_one(zL = zL, zU = zU)

  out <- newtraph(ders, dat = y, x0 = start,
                  quiet = TRUE, print_final = FALSE,
                  step_scale = 0.5, max_halving = 60)

  par_hat   <- as.numeric(out[[1]])  # (xi, logtheta)
  xi_hat    <- par_hat[1]
  theta_hat <- exp(par_hat[2])

  V <- out[[3]]                      # (-H)^{-1} at solution, for (xi, logtheta)
  se_xi <- sqrt(V[1, 1])

  list(
    xi_hat     = xi_hat,
    theta_hat  = theta_hat,
    logLik     = out[[2]],
    se_xi      = se_xi,
    ci_xi_wald = c(xi_hat - 1.96 * se_xi, xi_hat + 1.96 * se_xi),
    start      = start,
    z_bounds   = c(zL, zU),
    out        = out,
    fInf       = V
  )
}

# Fit separately
fit50 <- fit_one_newtraph(y50, zL = -100, zU = 100)
fit80 <- fit_one_newtraph(y80, zL = -100, zU = 100)
```

```r
cat("\n--- MLE (50m) ---\n"); print(fit50); cat("Wald CI xi (50m):\n"); print(fit50$ci_xi_wald)
cat("\n--- MLE (80m) ---\n"); print(fit80); cat("Wald CI xi (80m):\n"); print(fit80$ci_xi_wald)

# ============================================================
# Profile-LR CI for xi
# ============================================================

nll_one <- function(par, y, zL = -100, zU = 100) -logLik_one(par, y = y, zL = zL, zU = zU)

profile_logLik_xi <- function(xi_fixed, y, logtheta_center, zL = -100, zU = 100) {
  obj <- function(logtheta) nll_one(c(xi_fixed, logtheta), y = y, zL = zL, zU = zU)
  opt <- optimize(obj, interval = c(logtheta_center - log(20), logtheta_center + log(20)))
  list(logLik = -opt$objective, logtheta_hat = opt$minimum)
}

ci_xi_profile <- function(y, xi_hat, theta_hat, logLik_hat, zL = -100, zU = 100, alpha = 0.05) {
  cutoff <- qchisq(1 - alpha, df = 1)

  target <- function(xi) {
    ell_prof <- profile_logLik_xi(xi_fixed = xi, y = y,
                                  logtheta_center = log(theta_hat),
                                  zL = zL, zU = zU)$logLik
    2 * (logLik_hat - ell_prof) - cutoff
  }

  step0 <- max(0.25, 0.5 * theta_hat)

  left <- xi_hat - step0
  step <- step0
  for (k in 1:80) {
    tl <- target(left)
    if (is.finite(tl) && tl < 0) { left <- left - step; step <- step * 1.2 } else break
  }

  right <- xi_hat + step0
  step <- step0
  for (k in 1:80) {
    tr <- target(right)
    if (is.finite(tr) && tr < 0) { right <- right + step; step <- step * 1.2 } else break
  }

  lower <- uniroot(target, interval = c(left, xi_hat))$root
  upper <- uniroot(target, interval = c(xi_hat, right))$root
  c(lower, upper)
}

ci50_prof <- ci_xi_profile(y50, fit50$xi_hat, fit50$theta_hat, fit50$logLik, zL = -100, zU = 100)
ci80_prof <- ci_xi_profile(y80, fit80$xi_hat, fit80$theta_hat, fit80$logLik, zL = -100, zU = 100)
cat("\nProfile-LR CI xi (50m):\n"); print(ci50_prof)
cat("\nProfile-LR CI xi (80m):\n"); print(ci80_prof)

# ============================================================
# LRT: Reduced (common xi,theta) vs Full (separate per height)
```

```r
# ============================================================

logLik_full <- function(par, y50, y80, zL=-100, zU=100) {
  xi1 <- par[1]; th1 <- exp(par[2])
  xi2 <- par[3]; th2 <- exp(par[4])
  sum(log_m_vec_stable(y50, xi1, th1, zL=zL, zU=zU)) +
    sum(log_m_vec_stable(y80, xi2, th2, zL=zL, zU=zU))
}

logLik_reduced <- function(par, y50, y80, zL=-100, zU=100) {
  xi <- par[1]; th <- exp(par[2])
  sum(log_m_vec_stable(y50, xi, th, zL=zL, zU=zU)) +
    sum(log_m_vec_stable(y80, xi, th, zL=zL, zU=zU))
}

nll_full    <- function(par, ...) -logLik_full(par, ...)
nll_reduced <- function(par, ...) -logLik_reduced(par, ...)

fit_full <- function(y50, y80, zL=-100, zU=100) {
  starts <- rbind(
    c(fit50$xi_hat, log(fit50$theta_hat), fit80$xi_hat, log(fit80$theta_hat)),
    c(ev_start(y50), ev_start(y80))
  )

  best <- NULL
  best_val <- Inf

  for (s in 1:nrow(starts)) {
    opt <- optim(starts[s,], nll_full, y50=y50, y80=y80, zL=zL, zU=zU,
                 method="BFGS", hessian=TRUE, control=list(maxit=4000))
    if (is.finite(opt$value) && opt$value < best_val) {
      best_val <- opt$value
      best <- opt
    }
  }

  if (is.null(best)) stop("Full model optimization failed for all starts.")
  if (best$convergence != 0) warning("Full model did not report convergence.")

  list(
    xi50_hat = best$par[1], th50_hat = exp(best$par[2]),
    xi80_hat = best$par[3], th80_hat = exp(best$par[4]),
    logLik   = -best$value,
    opt      = best
  )
}

fit_reduced <- function(y50, y80, zL=-100, zU=100) {
  yall <- c(y50, y80)
  starts <- rbind(
    ev_start(yall),
    c(mean(yall), log(sd(yall)))
  )
```

```r
  best <- NULL
  best_val <- Inf

  for (s in 1:nrow(starts)) {
    opt <- optim(starts[s,], nll_reduced, y50=y50, y80=y80, zL=zL, zU=zU,
                 method="BFGS", hessian=TRUE, control=list(maxit=4000))
    if (is.finite(opt$value) && opt$value < best_val) {
      best_val <- opt$value
      best <- opt
    }
  }

  if (is.null(best)) stop("Reduced model optimization failed for all starts.")
  if (best$convergence != 0) warning("Reduced model did not report convergence.")

  list(
    xi_hat = best$par[1], th_hat = exp(best$par[2]),
    logLik = -best$value,
    opt    = best
  )
}

full <- fit_full(y50, y80, zL=-100, zU=100)
red  <- fit_reduced(y50, y80, zL=-100, zU=100)

LR   <- 2 * (full$logLik - red$logLik)
df   <- 2
pval <- 1 - pchisq(LR, df=df)

cat("\n--- LRT (Full vs Reduced) ---\n")
print(c(LR = LR, df = df, p_value = pval))

# ==============================================================
# Plot EV mixing density/densities for selected model
# (Select full if p < 0.05; otherwise reduced)
# ==============================================================

# Right-skew Gumbel density for mu
g_mu <- function(mu, xi, theta) {
  z <- (mu - xi) / theta
  (1/theta) * exp(-z - exp(-z))
}

use_full <- is.finite(pval) && (pval < 0.05)

if (use_full) {
  mu_grid <- seq(
    min(full$xi50_hat - 6*full$th50_hat, full$xi80_hat - 6*full$th80_hat),
    max(full$xi50_hat + 10*full$th50_hat, full$xi80_hat + 10*full$th80_hat),
    length.out = 800
  )

  plot(mu_grid, g_mu(mu_grid, full$xi50_hat, full$th50_hat),
```

```r
       type = "l", lwd = 2,
       xlab = expression(mu), ylab = "density",
       main = "EV mixing densities g(mu|xi,theta) - Full model")
  lines(mu_grid, g_mu(mu_grid, full$xi80_hat, full$th80_hat), lwd = 2, lty = 2)
  legend("topright", legend = c("50m", "80m"), lwd = 2, lty = c(1, 2), bty = "n")

} else {
  mu_grid <- seq(red$xi_hat - 6*red$th_hat, red$xi_hat + 10*red$th_hat, length.out = 800)
  plot(mu_grid, g_mu(mu_grid, red$xi_hat, red$th_hat),
       type = "l", lwd = 2,
       xlab = expression(mu), ylab = "density",
       main = "EV mixing density g(mu|xi,theta) - Reduced model")
}

# ============================================================
# Q2: E(mu) = xi + gamma*theta + delta-method CI
# ============================================================

gamma <- 0.5772156649015329

wald_ci_mean_mu <- function(fit) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V      <- fit$fInf   # cov for (xi, logtheta)

  eta_hat <- xi_hat + gamma * th_hat
  grad <- c(1, gamma * th_hat)  # wrt (xi, logtheta)

  se <- sqrt(drop(t(grad) %*% V %*% grad))
  ci <- eta_hat + c(-1, 1) * qnorm(0.975) * se
  list(eta_hat = eta_hat, se = se, ci95 = ci)
}

ci50_eta <- wald_ci_mean_mu(fit50)
ci80_eta <- wald_ci_mean_mu(fit80)
cat("\n--- Q2: E(mu) ---\n"); print(ci50_eta); print(ci80_eta)

# ============================================================
# Q3: Median of observed Y + delta-method CI
# ============================================================

FY_stable <- function(y, xi, theta, zL = -100, zU = 100) {
  integrand <- function(z) {
    pnorm(y - (xi + theta * z)) * exp(-z - exp(-z))
  }
  val <- integrate(integrand, lower = zL, upper = zU,
                   rel.tol = 1e-8, abs.tol = 0, subdivisions = 5000)$value
  pmin(pmax(val, 0), 1)
}

median_y <- function(xi, theta, zL = -100, zU = 100) {
  f <- function(y) FY_stable(y, xi, theta, zL, zU) - 0.5
```

```r
  y0 <- xi + gamma * theta
  lo <- y0 - 10 * theta - 5
  hi <- y0 + 10 * theta + 5

  for (k in 1:60) if (f(lo) > 0) lo <- lo - (2 + theta) else break
  for (k in 1:60) if (f(hi) < 0) hi <- hi + (2 + theta) else break

  uniroot(f, interval = c(lo, hi))$root
}

wald_ci_median_y <- function(fit, zL = -100, zU = 100) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V      <- fit$fInf

  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    median_y(xi, th, zL = zL, zU = zU)
  }

  par_hat <- c(xi_hat, log(th_hat))
  m_hat <- h(par_hat)

  g <- numDeriv::grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))
  ci <- m_hat + c(-1, 1) * qnorm(0.975) * se

  list(median_hat = m_hat, se = se, ci95 = ci)
}

ci50_med <- wald_ci_median_y(fit50, zL = -100, zU = 100)
ci80_med <- wald_ci_median_y(fit80, zL = -100, zU = 100)
cat("\n--- Q3: Median(Y) ---\n"); print(ci50_med); print(ci80_med)

# ==============================================================
# Q4: P(4 <= mu <= 17) + delta-method CI (clamped to [0,1])
# ==============================================================

G_mu <- function(u, xi, theta) exp(-exp(-(u - xi)/theta))

wald_ci_p_between_mu <- function(fit, a = 4, b = 17) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat
  V      <- fit$fInf

  h <- function(par) {
    xi <- par[1]
    th <- exp(par[2])
    G_mu(b, xi, th) - G_mu(a, xi, th)
  }

  par_hat <- c(xi_hat, log(th_hat))
```

```r
  p_hat <- h(par_hat)

  g <- numDeriv::grad(h, par_hat)
  se <- sqrt(drop(t(g) %*% V %*% g))

  ci <- p_hat + c(-1, 1) * qnorm(0.975) * se
  ci <- pmin(pmax(ci, 0), 1)

  list(p_hat = p_hat, se = se, ci95 = ci)
}

ci50_p <- wald_ci_p_between_mu(fit50, a = 4, b = 17)
ci80_p <- wald_ci_p_between_mu(fit80, a = 4, b = 17)
cat("\n--- Q4: P(4 <= mu <= 17) ---\n"); print(ci50_p); print(ci80_p)

# ============================================================
# Q6: Monte Carlo approximation to E(mu) + MC intervals
# ============================================================

set.seed(43)

rEV <- function(n, xi, theta) {
  u <- runif(n)
  xi - theta * log(-log(u))
}

mc_E_mu <- function(fit, M = 200000) {
  xi_hat <- fit$xi_hat
  th_hat <- fit$theta_hat

  mu_draws <- rEV(M, xi_hat, th_hat)

  mc_mean <- mean(mu_draws)
  mc_se   <- sd(mu_draws) / sqrt(M)
  mc_ci   <- mc_mean + c(-1, 1) * qnorm(0.975) * mc_se

  plug_in <- xi_hat + gamma * th_hat

  list(M = M, plug_in = plug_in, mc_mean = mc_mean, mc_se = mc_se, mc_ci95 = mc_ci)
}

mc50 <- mc_E_mu(fit50, M = 200000)
mc80 <- mc_E_mu(fit80, M = 200000)
cat("\n--- Q6: Monte Carlo E(mu) ---\n"); print(mc50); print(mc80)

# ============================================================
# End appendix chunk
# ============================================================
```